

MEESHO – WOMEN’S FASHION E-COMMERCE ORDER MANAGEMENT SYSTEM

CHAPTER – 5

PRODUCT AND CATEGORY MANAGEMENT SYSTEM

1. Introduction

The Meesho Women’s Fashion E-Commerce Order Management System is designed to manage customer, seller, product, category, order, payment and delivery information efficiently. The system uses related tables to store and organize product and order details. Primary key and foreign key constraints are used to maintain data integrity and establish relationships between different entities.

SQL operations such as INSERT, UPDATE and DELETE are performed to manage product and order information. Reports can be generated using SQL and PL/SQL to analyze products, orders, stock and category-wise information.

2. Objectives

- Design tables for Customer, Seller, Category, Product, Order, Payment and Delivery entities.
- Define primary key and foreign key relationships.
- Store customer, seller, product and category details.
- Manage product price and stock information.
- Store and manage customer orders.
- Maintain payment and delivery details.
- Perform product and order insertion, updating and deletion.
- Generate category-wise and order-wise reports using SQL and PL/SQL.

3. Table Design

3.1 Customer Table

Attribute	Data Type	Key	Description
Customer_ID	NUMBER	Primary Key	Unique customer identifier
Customer Name	VARCHAR2(100)	-	Customer name
Email	VARCHAR2(100)	Unique	Customer email address
Phone	VARCHAR2(15)	Unique	Customer phone number
Address	VARCHAR2(200)	-	Customer delivery address

3.2 Seller Table

Attribute	Data Type	Key	Description
-----------	-----------	-----	-------------

Seller_ID	NUMBER	Primary Key	Unique seller identifier
Seller_Name	VARCHAR2(100)	-	Seller name
Email	VARCHAR2(100)	Unique	Seller email address
Phone	VARCHAR2(15)	Unique	Seller phone number

3.3 Category Table

Attribute	Data Type	Key	Description
Category_ID	NUMBER	Primary Key	Unique category identifier
Category_Name	VARCHAR2(100)	-	Product category name

3.4 Product Table

Attribute	Data Type	Key	Description
Product_ID	NUMBER	Primary Key	Unique product identifier
Product_Name	VARCHAR2(100)	-	Product name
Price	NUMBER(10,2)	-	Product price
Stock	NUMBER	-	Available quantity
Seller_ID	NUMBER	Foreign Key	References Seller(Seller_ID)
Category_ID	NUMBER	Foreign Key	References Category(Category_ID)

3.5 Order Table

Attribute	Data Type	Key	Description
Order_ID	NUMBER	Primary Key	Unique order identifier
Customer_ID	NUMBER	Foreign Key	References Customer(Customer_ID)
Product_ID	NUMBER	Foreign Key	References Product(Product_ID)
Quantity	NUMBER	-	Quantity ordered
Order_Date	DATE	-	Date of order
Order_Status	VARCHAR2(50)	-	Current order status

3.6 Payment Table

Attribute	Data Type	Key	Description
Payment_ID	NUMBER	Primary Key	Unique payment identifier
Order_ID	NUMBER	Foreign Key	References Order(Order_ID)
Payment_Method	VARCHAR2(50)	-	Mode of payment
Payment_Status	VARCHAR2(50)	-	Payment status

Amount	NUMBER(10,2)	-	Payment amount
--------	--------------	---	----------------

3.7 Delivery Table

Attribute	Data Type	Key	Description
Delivery_ID	NUMBER	Primary Key	Unique delivery identifier
Order_ID	NUMBER	Foreign Key	References Order(Order_ID)
Delivery_Date	DATE	-	Expected delivery date
Delivery_Status	VARCHAR2(50)	-	Current delivery status

4. Relationship

The Meesho database contains relationships between customers, sellers, categories, products, orders, payments and deliveries.

- One customer can place many orders.
- One seller can sell many products.
- One category can contain many products.
- One product can appear in many orders.
- Each order is associated with one payment.
- Each order is associated with one delivery.

CUSTOMER (1) ————— (M) ORDER

SELLER (1) ————— (M) PRODUCT

CATEGORY (1) ————— (M) PRODUCT

PRODUCT (1) ————— (M) ORDER

ORDER (1) ————— (1) PAYMENT

ORDER (1) ————— (1) DELIVERY

5. SQL Table Creation

```
SQL> CREATE TABLE Meesho_Product (
  2     Product_ID INT PRIMARY KEY,
  3     Product_Name VARCHAR(100) NOT NULL,
  4     Category_ID INT,
  5     Price NUMBER(10,2) NOT NULL,
  6     Stock_Quantity INT NOT NULL,
  7     CONSTRAINT fk_meesho_category
  8     FOREIGN KEY (Category_ID)
  9     REFERENCES Meesho_Category(Category_ID)
 10 );
```

Table created.

6. Insert Records

```
SQL> INSERT INTO Meesho_Category
  2 VALUES (1, 'Sarees', 'Traditional and designer sarees');

1 row created.

SQL>
SQL> INSERT INTO Meesho_Category
  2 VALUES (2, 'Kurtis', 'Women ethnic kurtis');

1 row created.

SQL>
SQL> INSERT INTO Meesho_Category
  2 VALUES (3, 'Western Wear', 'Modern western fashion');

1 row created.

SQL>
SQL> INSERT INTO Meesho_Category
  2 VALUES (4, 'Jewellery', 'Fashion jewellery and accessories');

1 row created.
```

```
SQL> INSERT INTO Meesho_Product
  2 VALUES (101, 'Cotton Saree', 1, 599.00, 25);

1 row created.

SQL>
SQL> INSERT INTO Meesho_Product
  2 VALUES (102, 'Printed Kurti', 2, 499.00, 30);

1 row created.

SQL>
SQL> INSERT INTO Meesho_Product
  2 VALUES (103, 'Western Top', 3, 399.00, 20);

1 row created.

SQL>
SQL> INSERT INTO Meesho_Product
  2 VALUES (104, 'Fashion Earrings', 4, 199.00, 40);

1 row created.

SQL>
SQL> INSERT INTO Meesho_Product
  2 VALUES (105, 'Designer Kurti', 2, 799.00, 15);

1 row created.

SQL>
SQL> COMMIT;|
```

```
SQL> SELECT * FROM Meesho_Category;
```

CATEGORY_ID	CATEGORY_NAME	DESCRIPTION
1	Sarees	Traditional and designer sarees

2	Kurtis	Women ethnic kurtis
---	--------	---------------------

CATEGORY_ID	CATEGORY_NAME	DESCRIPTION
3	Western Wear	Modern western fashion

4	Jewellery	Fashion jewellery and accessories
---	-----------	-----------------------------------

7. Product and Order Updating and Deletion

Product Updating

```
SQL> UPDATE Meesho_Product
  2  SET Price = 649.00,
  3      Stock_Quantity = 30
  4  WHERE Product_Name = 'Cotton Saree';

1 row updated.

SQL>
SQL> COMMIT;

Commit complete.

SQL> UPDATE Meesho_Product
  2  SET Price = 549.00,
  3      Stock_Quantity = 35
  4  WHERE Product_Name = 'Printed Kurti';

1 row updated.

SQL>
SQL> COMMIT;|
```

Product Deletion

```
SQL> DELETE FROM Meesho_Product
  2  WHERE Product_Name = 'Western Top';

1 row deleted.

SQL>
SQL> COMMIT;

Commit complete.

SQL> DELETE FROM Meesho_Product
  2  WHERE Product_Name = 'Fashion Earrings';

1 row deleted.

SQL>
SQL> COMMIT;|
```

8. Category-wise Product Report (SQL)

```
SQL> SELECT C.Category_Name,
2         P.Product_ID,
3         P.Product_Name,
4         P.Price,
5         P.Stock_Quantity
6 FROM Meesho_Category C
7 JOIN Meesho_Product P
8 ON C.Category_ID = P.Category_ID
9 ORDER BY C.Category_Name;
```

CATEGORY_NAME				
PRODUCT_ID				
PRODUCT_NAME				
PRICE STOCK_QUANTITY				
Kurtis				
	102			
Printed Kurti	549		35	
CATEGORY_NAME				
PRODUCT_ID				
PRODUCT_NAME				
PRICE STOCK_QUANTITY				
Kurtis				
	105			
Designer Kurti	799		15	
CATEGORY_NAME				
PRODUCT_ID				
PRODUCT_NAME				
PRICE STOCK_QUANTITY				
Sarees				
	101			
Cotton Saree	649		30	

9. PL/SQL Program – Category-wise Report

```
SQL> SELECT P.Product_ID,
2         P.Product_Name,
3         C.Category_Name,
4         P.Price
5 FROM Meesho_Product P
6 JOIN Meesho_Category C
7 ON P.Category_ID = C.Category_ID
8 WHERE C.Category_Name IN ('Sarees', 'Kurtis', 'Western Wear');
```

PRODUCT_ID				
PRODUCT_NAME				
CATEGORY_NAME				
PRICE				
102				
Printed Kurti				
Kurtis	549			
PRODUCT_ID				
PRODUCT_NAME				
CATEGORY_NAME				
PRICE				
105				
Designer Kurti				
Kurtis	799			
PRODUCT_ID				
PRODUCT_NAME				
CATEGORY_NAME				
PRICE				
101				
Cotton Saree				
Sarees	649			

10. PL/SQL Program – Product Price Update

```
SQL> SELECT C.Category_Name,  
2      COUNT(P.Product_ID) AS Product_Count  
3 FROM Meesho_Category C  
4 LEFT JOIN Meesho_Product P  
5 ON C.Category_ID = P.Category_ID  
6 GROUP BY C.Category_Name;
```

CATEGORY_NAME

PRODUCT_COUNT

Sarees

1

Kurtis

2

Jewellery

0

CATEGORY_NAME

PRODUCT_COUNT

Western Wear

11. Expected Category-wise Report

Category	Total Products	Total Stock
Sarees	1	50
Kurtis	1	40
Western Wear	1	30

The report displays the number of products and total available stock under each category.

12. Conclusion

The Meesho Women's Fashion E-Commerce Order Management System successfully represents the relationships between customers, sellers, categories, products, orders, payments and deliveries. Primary key and foreign key constraints help maintain data integrity and establish relationships between the tables.

INSERT, UPDATE and DELETE operations support effective product and order management. SQL reports provide useful information about products and stock, while PL/SQL programs demonstrate procedural database programming for report generation and product price updates.